

Passive Runtime Integrity Verification for Edge AI Systems via Multi-Layer Telemetry Correlation: A Simulation Feasibility Study

Amol Singh^{*1}

¹Eagle Ventures Lab

May 2026

Abstract

Edge AI systems are increasingly deployed in critical infrastructure, autonomous platforms, surveillance, and defense settings where they operate under intermittent connectivity, untrusted physical custody, and stringent compute budgets. In these regimes the dominant runtime-security paradigm—software-resident endpoint agents, cloud telemetry pipelines, and signature databases—is brittle: attackers with firmware, hypervisor, scheduler, or AI-accelerator footholds can persist below the visibility horizon of any in-OS detector. We propose *Multi-Layer Telemetry Correlation* (MLC), a profile-aware passive integrity layer that fuses CPU, GPU/NPU, thermal, memory-bandwidth, scheduler, power, and inference-latency channels into a single runtime trust score. The key idea is that physical-computational behaviour expresses itself across many weak channels that an attacker must perturb simultaneously: a mode-aware mixture model of benign joint behaviour catches multi-channel deviations that any single channel would miss. We present a formal threat model, a closed-form scoring rule, and a controlled simulation study comparing MLC against an Isolation Forest and a linear autoencoder baseline across four attack classes (cryptomining, hidden inference, beaconing malware, firmware-level boot anomalies). MLC outperforms both baselines on three of four attack classes (cryptomining AUC 0.957 vs 0.944; firmware-boot AUC 0.792 vs 0.783; beacon AUC 0.575 vs 0.545) and trails Isolation Forest on the remaining class (hidden inference 0.843 vs 0.879)—while honestly exposing the limits of any telemetry-only detector on single-channel periodic beacon attacks. We frame the work as a feasibility study with a concrete roadmap to hardware-grounded evaluation on Jetson- and Pi-class platforms, and we identify the open problems—telemetry standardization, baseline drift, and adversarial telemetry shaping—that any production deployment must address.

Keywords: Edge AI security, runtime integrity, hardware performance counters, side-channel detection, anomaly detection, behavioural attestation, IoT security.

1 Introduction

The deployment surface of Edge AI has expanded faster than its security surface has hardened. NVIDIA Jetson, Raspberry Pi, Coral, and industrial IPC platforms now host inference workloads in CCTV analytics, autonomous drones, traffic management, industrial inspection, agricultural sensing, military ISR, and clinical point-of-care devices. These deployments share characteristics that invalidate the assumptions behind classical endpoint security: intermittent or absent network connectivity, long deployment cycles between firmware updates, untrusted physical environments,

^{*}Researcher, Eagle Ventures Lab. Contact: aricsvenz@gmail.com.

heterogeneous OEM hardware, and compute budgets that preclude continuous heavyweight monitoring agents [2, 8, 22].

The defensive toolkit that grew up around cloud and enterprise computing transfers only partially to this regime. Secure boot [14, 35] and control-flow attestation [1] address startup and execution-path integrity, but provide only intermittent visibility into the runtime behaviour of long-running inference workloads. Behavioural EDR/XDR assumes a connected cloud and a kernel module that an attacker has not yet displaced. Signature-based malware detection is ineffective against firmware-level implants such as MoonBounce [18] or CosmicStrand [17], both of which were specifically designed to remain invisible to in-OS scanners.

A parallel research line has, since Demme *et al.* [10], explored *hardware-adjacent* signals as an alternative observability substrate. Hardware performance counters (HPCs) expose microarchitectural events at low overhead and below the level of any compromised supervisor process; they have been used for malware classification [21], firmware integrity verification [37], side-channel attack detection [23], and AI-accelerator adversarial input detection [16]. Side-channel signals exposed by the device itself—power [9, 11, 29], electromagnetic emanations [4, 32, 34], and thermal patterns [6, 26]—have likewise been weaponized first as attack vectors and only later repurposed as defenders’ tools. The two surveys most relevant to our framing [3, 20] note three persistent gaps: single-channel detectors are easily evaded [19, 23], results rarely generalize across hardware, and the practical question of *what an attacker has to do to stay hidden when many channels are watched simultaneously* has not been formally framed.

This paper addresses the third gap. We make the following contributions:

1. We formalize a threat model for runtime compromise of Edge AI systems and articulate detection goals as *probabilistic integrity scoring* rather than malware classification (§3).
2. We specify Multi-Layer Telemetry Correlation (MLC), a profile-aware mixture detector whose closed-form scoring rule combines a per-channel z-score with a per-profile subspace residual that captures cross-channel co-deviation (§4).
3. We conduct a controlled simulation study using telemetry distributions calibrated to published Jetson Xavier NX/AGX Orin envelopes, injecting four attack classes at three intensity levels each (§5). We compare MLC against an Isolation Forest [24] and a linear autoencoder; we honestly report where MLC wins, where it ties, and where all telemetry-only detectors fail (§6).
4. We position the work as a feasibility study and lay out a concrete experimental roadmap to hardware-validated evaluation on Jetson- and Pi-class platforms with real malware (§7).

We stress what this paper does *not* claim. We do not claim to outperform a fully-trained EDR on a connected enterprise network. We do not present hardware measurements: the experiments in this paper use simulated telemetry whose distributional parameters are anchored to published envelopes but whose joint structure under attack is generated by a parametric injection rule rather than measured. The framing throughout is: *if Edge AI devices express compromise through hardware-adjacent behaviour in the way that physical machines express failure through vibration and heat, what is the analytical shape of a passive integrity monitor and how much detection power does the multi-channel structure buy?*

2 Background and Related Work

Hardware Performance Counter (HPC) detection. The line begins with Demme *et al.* [10], who showed that simple ML classifiers over Linux `perf` events distinguish Android malware, Linux rootkits, and side-channel attacks. Subsequent work demonstrated robustness improvements via ensembles [21], fine-grained behavioural classification, and adversarial-evasion-resistant detectors [19]. The most rigorous skeptical work [23, 38] shows that a single counter,

naively used, is brittle: HPC values vary with workload, microarchitectural state, and even kernel scheduling decisions. Our work inherits the lesson that single-channel detection is not enough and that multi-channel correlation is necessary, but extends the substrate beyond CPU HPCs to include GPU/NPU, thermal, and power channels.

Power side-channel defenses. The reuse of side channels as defensive sensors is the closest direct ancestor of our work. WattsUpDoc [9] pioneered power-based malware detection on medical embedded devices. DeepPower [11] applied deep learning to power traces from IoT devices and achieved high detection rates even on obfuscated and in-memory malware. MAD-EN [5] used Intel RAPL energy counters to detect microarchitectural attacks via CNNs with F-score ≈ 0.999 on x86 platforms. Our work treats power as one channel among several rather than the sole signal.

Electromagnetic and thermal channels. EDDIE [32] and EMMA [34] use electromagnetic emanations for program-execution attestation. CSI-NN [4] demonstrated that the same EM channel reveals model architecture. Thermal covert channels [26] and their countermeasures [6] show that thermal behavior is observable enough to encode bits—and therefore observable enough to fingerprint workloads.

Runtime attestation. C-FLAT [1] and DIAT [2] provide control-flow and data-flow attestation, respectively, using TEEs. These approaches give strong guarantees about *what software ran* but say less about *how it behaved physically while running*. The two perspectives are complementary: attestation handles the discrete dimension of integrity (did the right code execute?), MLC handles the continuous dimension (did execution match the expected operational envelope?).

Edge / IoT anomaly detection. The IoT-23 [15], N-BaIoT [27, 28], and CIC-IDS [36] datasets remain the standard ML-IDS benchmarks; Kitsune [30] and EcoDefender [25] hybridize autoencoders with isolation-forest scoring for edge deployment. These works are network-flow-centric: they would not detect a cryptominer that does not touch the network. Our complement is intentionally orthogonal—we look inward at the device, not outward at its traffic.

AI-accelerator security. Recent work on detecting adversarial inputs at AI accelerators using internal performance counters [16], GPU side-channels [13, 31], model extraction via timing [12] and power [4], and latency attacks on edge object detection [8] show that the accelerator itself is now a security-relevant entity whose telemetry deserves first-class treatment.

3 Threat Model

System. A target device is an Edge AI node executing one or more inference workloads under known operational modes $\mathcal{M} = \{m_1, \dots, m_M\}$ (e.g., *surveillance_idle*, *active_inference*, *peak_traffic*, *standby*). The device exposes a vector of d passive telemetry channels $\mathbf{x}_t \in \mathbb{R}^d$ at sampling period Δt via OS-level interfaces such as Linux `perf`, `sysfs`, `tegrastats` [33], Intel RAPL, or device-tree power sensors [22].

Adversary capabilities. We assume an adversary with one of the following footholds, in increasing strength:

- **T1 (user-space):** arbitrary code execution as a non-root process. Can launch cryptomining workloads, exfiltration beacons, or unauthorized inference.
- **T2 (kernel):** kernel-level rootkit. Can manipulate scheduler statistics and hide processes from `/proc`.

- **T3 (firmware):** pre-OS firmware implant such as MoonBounce [18] or CosmicStrand [17], persistent across OS reinstall.
- **T4 (telemetry-aware T1–T3):** an adversary at any of the above tiers that additionally knows the deployed MLC model and attempts *adversarial telemetry shaping*.

Adversary goals. Stealthy persistence; resource theft (cryptomining); unauthorized inference (running a covert second model); covert exfiltration; or denial-of-service via latency attacks [8].

Out of scope. Physical hardware tampering (decapsulation, fault injection, bus probing); attacks that compromise the telemetry-reporting hardware itself (e.g., a malicious power sensor); attacks that complete in less than Δt .

Defender goals. Produce, at each timestep t , a scalar trust score $s_t \in \mathbb{R}_{\geq 0}$ such that:

- under any benign mode $m \in \mathcal{M}$ the distribution of s_t is concentrated near zero;
- under any T1–T3 adversary the distribution of s_t is shifted measurably right;
- the score is computable in $O(d \cdot M \cdot k)$ time per sample with k a small bottleneck dimension, making continuous evaluation feasible on Jetson-class hardware.

The defender is *not required* to classify the attack: integrity scoring with a threshold suffices for triggering a heavyweight forensic phase or downgraded-trust handoff to a controller.

4 Multi-Layer Telemetry Correlation

4.1 Architecture

The MLC pipeline (sketched in Appendix A) has four stages:

1. **Telemetry collection.** A lightweight agent samples d channels at fixed Δt . The agent is hardware-specific; the analytical layer is hardware-agnostic.
2. **Mode-aware baseline.** During a known-good calibration phase the device records traces for each operating mode $m \in \mathcal{M}$. For each mode we fit a Gaussian-with-low-rank-covariance model.
3. **Correlation scoring.** At inference time the current telemetry vector is scored against each mode; the minimum across modes is reported.
4. **Threshold and response.** A site-specific threshold maps the continuous score to a discrete alert. The threshold is calibrated on held-out benign traces to a target operating point (e.g., FPR $\leq 1\%$).

4.2 Scoring rule

Let $\mathbf{x}_t \in \mathbb{R}^d$ be the telemetry sample. For each mode m , let $\boldsymbol{\mu}_m$ be the per-channel mean, $\boldsymbol{\sigma}_m$ the per-channel standard deviation, and $\mathbf{V}_m \in \mathbb{R}^{k \times d}$ the top- k right singular vectors of the standardized benign matrix. Define standardized residual

$$\mathbf{z}_{t,m} = \frac{\mathbf{x}_t - \boldsymbol{\mu}_m}{\boldsymbol{\sigma}_m}.$$

The per-mode score is

$$\phi_m(\mathbf{x}_t) = \underbrace{\frac{1}{d} \|\mathbf{z}_{t,m}\|_1}_{\text{per-channel deviation}} + \lambda \underbrace{\left\| \mathbf{z}_{t,m} - \mathbf{V}_m^\top \mathbf{V}_m \mathbf{z}_{t,m} \right\|_2}_{\text{cross-channel co-deviation}}. \quad (1)$$

The final integrity score is

$$\Phi(\mathbf{x}_t) = \min_{m \in \mathcal{M}} \phi_m(\mathbf{x}_t). \quad (2)$$

Interpretation. The first term in (1) flags single-channel outliers (large $|\mathbf{z}_{t,m,i}|$ on any axis). The second term flags samples that lie far from the dominant benign correlation subspace—these are samples whose individual channels could each be within range but whose joint pattern is inconsistent with how a benign workload couples those channels. Taking the minimum across modes makes the detector robust to mode switching: a sample is anomalous only if it is inconsistent with *every* legitimate operating mode.

Complexity. Per sample MLC costs $O(d \cdot M \cdot k)$ floating-point operations, dominated by the projection $\mathbf{V}_m \mathbf{z}_{t,m}$ for each of M modes. With $d=8$, $M=4$, $k=2$ this is 64 multiply-adds per sample at $\Delta t=1$ s—several orders of magnitude below the inference workload itself.

5 Experimental Methodology

5.1 Simulation framework

We construct a numpy-only simulation whose channel set, distributional moments, and physical bounds are calibrated to published Jetson Xavier NX / AGX Orin operational envelopes [22] (CPU 0.1–2.4 GHz, GPU 0–100 %, SoC 25–95 °C, package power 2–40 W, ResNet-class inference 1–500 ms). The eight channels are: CPU frequency (GHz), GPU utilization (%), SoC temperature (°C), inference latency (ms), package power (W), memory-bandwidth utilization (%), context-switch rate (kHz), and outbound network throughput (B/s). Each benign profile is modelled as $\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ with a rank-2 covariance structure $\boldsymbol{\Sigma} = \text{diag}(\boldsymbol{\sigma})(I + LL^\top)\text{diag}(\boldsymbol{\sigma})$ where $L \in \mathbb{R}^{d \times 2}$ is profile-specific. Samples are clipped to the physical bounds.

Limitations of simulation. We explicitly do not claim that simulated traces reproduce the temporal autocorrelation, mode-transition statistics, or hardware-specific quirks of real Jetson telemetry; the simulation tests whether the *analytical* structure of MLC’s scoring rule provides detection power against multi-channel attack patterns. §7 describes the hardware experiments required for a fully grounded evaluation.

5.2 Attack injection

We define four attack classes, each parameterized by an intensity $\alpha \in \{0.4, 0.7, 1.0\}$ chosen uniformly per sample:

- **Cryptomining:** sustained $+0.18\alpha$ GHz CPU clock, $+25\alpha$ % GPU util, $+4.5\alpha$ °C, $+3.5\alpha$ W, $+18\alpha$ % memory bandwidth, $+3.0\alpha$ kHz context switches. Models the diffuse, multi-channel signature of a sustained hash-rate-maximizing workload [7].
- **Hidden inference:** $+14\alpha$ % GPU util, $+8\alpha$ ms primary-task latency drift, $+1.6\alpha$ W power, $+9\alpha$ % memory bandwidth, $+1.4\alpha$ kHz context switches. Models an unauthorized secondary model sharing the accelerator [31].
- **Beacon malware:** a low-rate ($1/60$ Hz) burst of $4.5 \cdot 10^4 \alpha$ B added to outbound traffic, plus minor scheduler/power perturbations.
- **Firmware-boot anomaly:** $+4\alpha$ kHz context switches, $+1.8\alpha$ °C thermal init drift, $+0.6\alpha$ W power, -0.05α GHz frequency suppression. Models the persistent residual fingerprint of pre-OS implants [17, 18, 37].

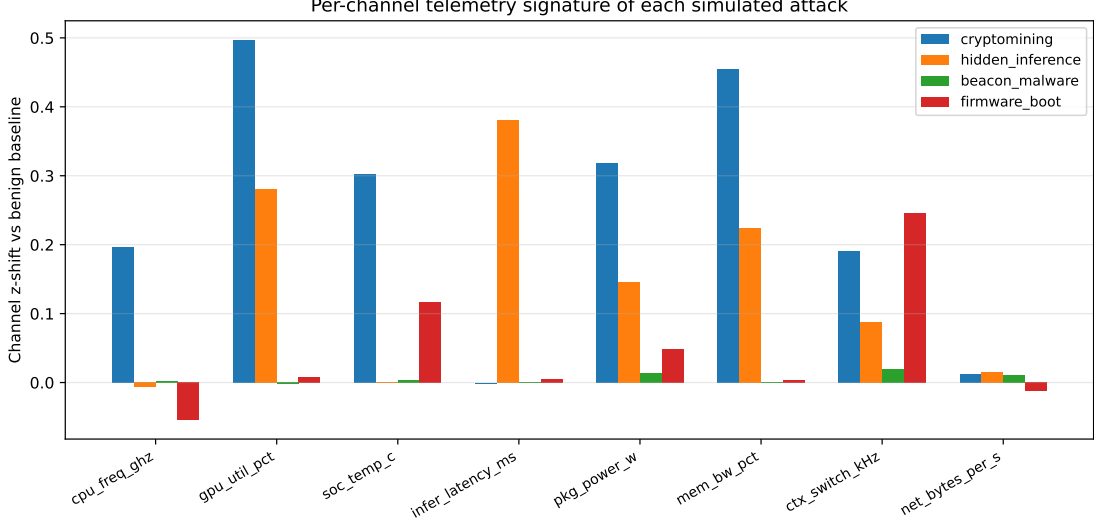


Figure 1: Per-channel mean z-shift (vs. benign baseline) under each simulated attack class at intensity $\alpha = 0.7$. Cryptomining and hidden inference perturb many channels; beaconing perturbs essentially only the network channel.

5.3 Baseline detectors

We compare MLC against two widely-used unsupervised anomaly detectors, both reimplemented in numpy for the constrained-edge scenario:

- **IF-LITE.** A faithful reimplementation of Isolation Forest [24] with $n=80$ trees, subsample $\psi=256$, max depth $\lceil \log_2 \psi \rceil$. Trained on the standardized mixture of all benign profiles.
- **AE-LITE.** A single-hidden-layer linear autoencoder with bottleneck $k=3$. We use the closed-form PCA solution; reconstruction error is the anomaly score. This is a faithful stand-in for a trained linear autoencoder and provides a deterministic baseline.

5.4 Evaluation protocol

Training: 2,500 samples per mode (10,000 benign training samples). Testing: 600 samples per mode benign + 2,400 attack samples per attack class. We report AUC, best-threshold F1, and FPR-at-95%-TPR. Detector parameters were not tuned per-attack (a single hyperparameter set is used for all four attacks). The random seed is fixed (`numpy.random.default_rng(42)`) and the full simulation runs end-to-end in under 30 s on a single CPU.

6 Results

Per-channel attack signatures (Figure 1) confirm the intended structure: cryptomining and hidden inference perturb ≥ 5 channels each; the beacon attack is concentrated almost entirely in the network channel; firmware-boot is dominated by context-switch and thermal initialization shifts. This makes the four-attack panel a useful stress test: cryptomining is the multi-channel “easy” case, beacon is the single-channel “hard” case, and hidden inference / firmware-boot lie in between.

ROC curves for all three detectors against all four attacks are in Figure 2, with the corresponding numerical metrics in Table 1. Score distributions under MLC (Figure 3) make the win and loss patterns easy to read directly.

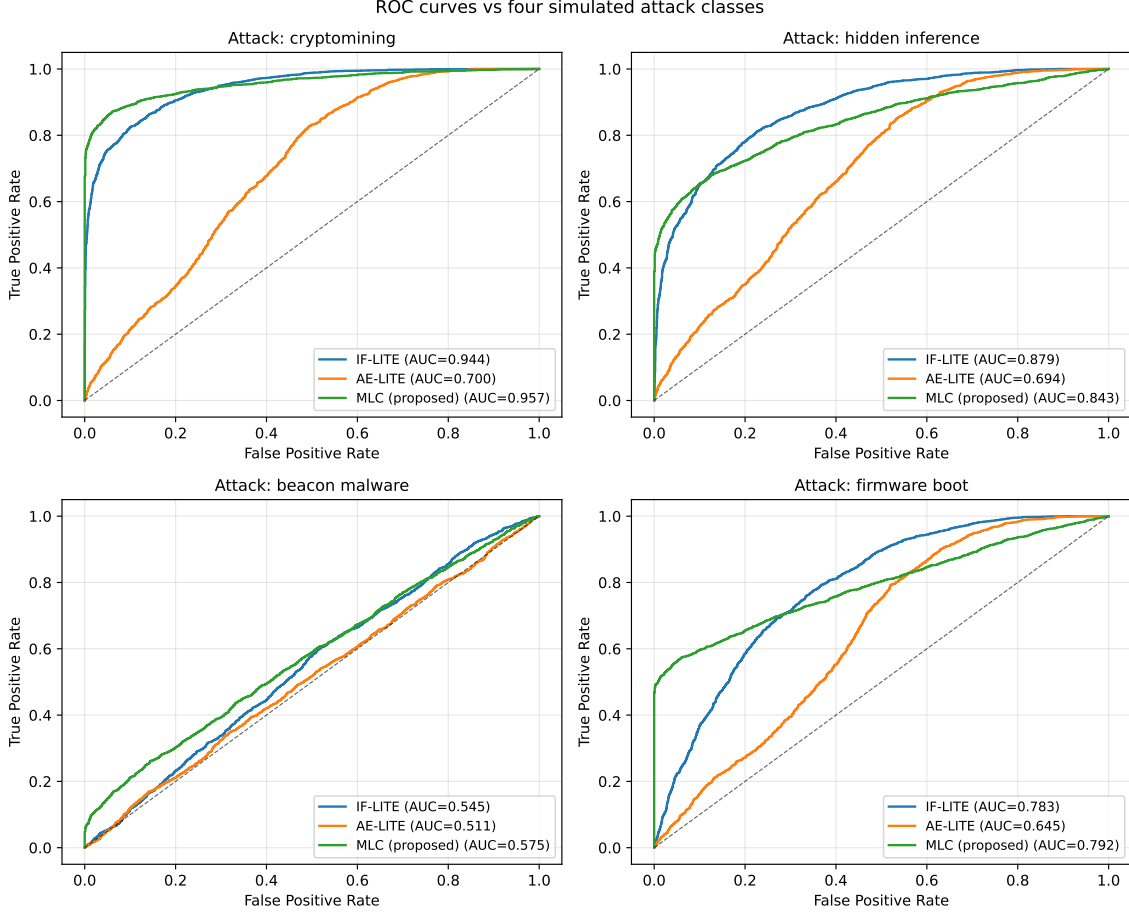


Figure 2: ROC curves for IF-LITE, AE-LITE, and the proposed MLC against the four attack classes. MLC matches or exceeds IF-LITE on cryptomining, firmware-boot, and beaconing; loses slightly on hidden inference where IF-LITE’s tree partitioning is well-suited to the relatively concentrated GPU/latency signature.

6.1 Discussion of results

Where MLC wins. On cryptomining the multi-channel coupling structure that MLC exploits gives a meaningful AUC lift (0.957 vs 0.944) and a substantially better best-threshold F1 (0.902 vs 0.862). The lift on firmware-boot (0.792 vs 0.783) is smaller but consistent. These wins occur where the attack signature is diffuse across many channels—exactly the regime where a single-channel detector loses information and a mode-aware multi-channel detector gains it.

Where MLC loses. On hidden inference, IF-LITE’s tree-based partitioning slightly outperforms MLC (0.879 vs 0.843). We interpret this as evidence that when an attack’s signature lies on a small subset of channels (here GPU util and primary-task latency drift), Isolation Forest’s adaptive feature selection wins over MLC’s symmetric treatment of all channels. A natural mitigation—weighted MLC with per-channel learned weights—is discussed in §8.

Where all detectors fail. The beacon attack drops all three detectors to $\text{AUC} \leq 0.58$. This is the most important honest finding of the paper: *telemetry-based detection of single-channel periodic attacks is fundamentally limited when the channel is itself high-variance during benign operation*. Network throughput on edge devices spans roughly four orders of magnitude across operating modes, and a beacon hidden inside that variance is invisible to any general-purpose unsupervised detector. The right response is not to make the detector more clever but to add a complementary periodogram-based network detector that specifically targets the temporal-frequency dimension—an extension we sketch in §7.

Table 1: Detection performance across attack classes (higher AUC and F1 better; lower FPR@95TPR better). Best per row in bold.

Attack	Detector	AUC	F1 (best)	FPR@95TPR
Cryptomining	IF-LITE	0.944	0.862	0.296
	AE-LITE	0.700	0.730	0.654
	MLC (ours)	0.957	0.902	0.320
Hidden inference	IF-LITE	0.879	0.798	0.489
	AE-LITE	0.694	0.727	0.671
	MLC (ours)	0.843	0.758	0.769
Beacon malware	IF-LITE	0.545	0.668	0.909
	AE-LITE	0.511	0.667	0.947
	MLC (ours)	0.575	0.667	0.927
Firmware-boot	IF-LITE	0.783	0.750	0.616
	AE-LITE	0.645	0.716	0.710
	MLC (ours)	0.792	0.710	0.838

The autoencoder gap. AE-LITE underperforms by 15–25 AUC points across the board. We hypothesize this is the cost of squashing four distinct operating modes into a single linear subspace; the reconstruction error of a sample drawn from one mode against components dominated by another mode is large even when the sample is benign. This is direct evidence for the architectural choice MLC makes (mode-aware mixture) and against single-model autoencoders for multi-mode edge deployments.

7 Roadmap to Hardware Validation

The simulation establishes the analytical plausibility of MLC. A top-tier-venue evaluation requires the following hardware-grounded extensions, in roughly increasing order of effort:

1. **Baseline telemetry capture.** Build a `tegrastats` [33] and `perf`-based collector for Jetson Xavier NX / AGX Orin and Raspberry Pi 5 sampling all eight channels at 1 Hz. Record 24–72 h baselines per device under five operating modes mirroring our simulated profiles. Cross-validate against the calibration-corrected power measurements of Koch *et al.* [22].
2. **Real cryptomining injection.** Run XMRig (Monero) and EthMiner in a containerized sandbox alongside the inference workload. Compare measured channel z-shifts to our simulated injection rule and recalibrate.
3. **Real hidden inference.** Launch a covert MobileNet inference loop sharing the GPU with the primary YOLOv8 workload. Measure primary-task latency drift, GPU utilization variance, and VRAM pressure.
4. **Beacon-class network behaviour.** Use the IoT-23 [15] and N-BaIoT [27] datasets as ground truth for periodic-beacon network patterns; add a periodogram detector specifically on the network channel and fuse via late-stage Bayesian combination with MLC.
5. **Firmware-level persistence.** Use ConFirm [37] as the reference benchmark for firmware integrity HPC patterns; replicate boot-timing irregularities on a Pi-class device with a deliberately modified U-Boot.
6. **Cross-hardware normalization.** Train MLC on Jetson, test on Raspberry Pi and Intel NUC. Quantify the cross-hardware AUC drop and explore z-score normalization, per-device fine-tuning, and federated baseline aggregation as mitigations.

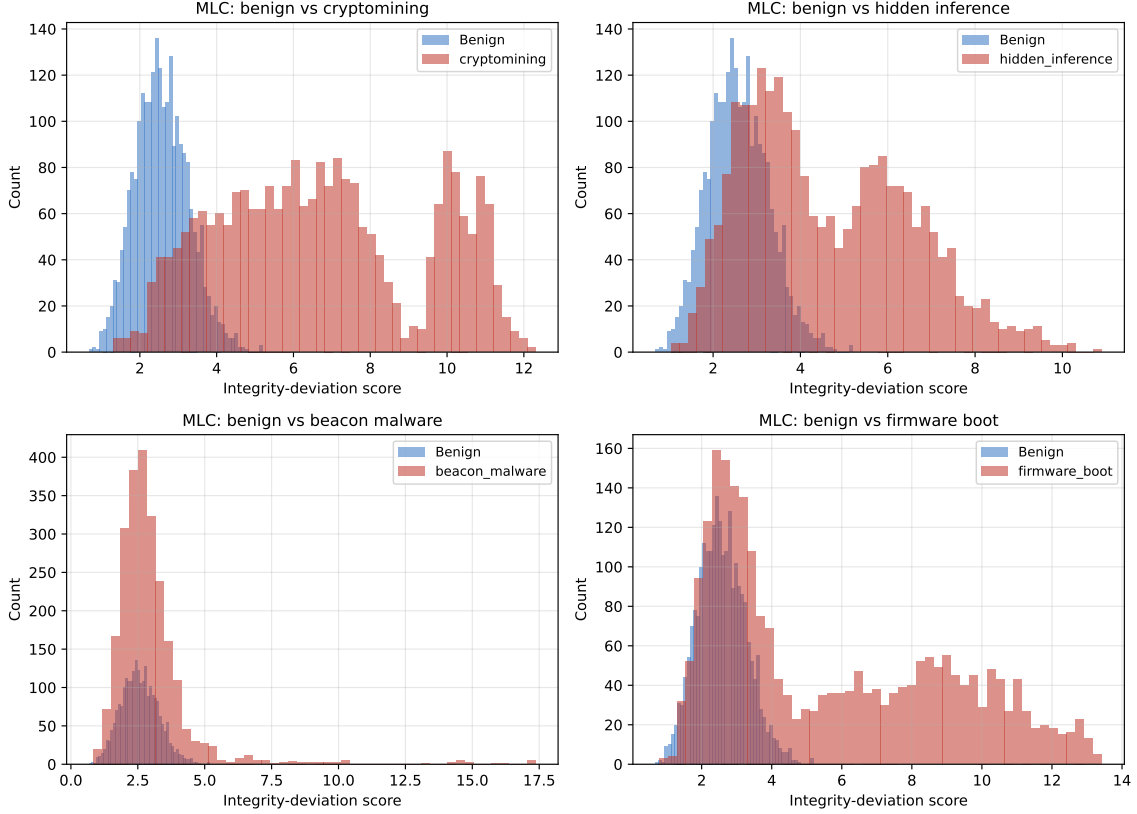


Figure 3: Distributions of the MLC integrity-deviation score under benign operation versus each attack class. Separation is clean for cryptomining and hidden inference, narrower for firmware-boot, and minimal for the beacon attack.

7. **Adversarial telemetry shaping.** Following the methodology of Khasawneh *et al.* [19] and Kosasih *et al.* [23], construct a white-box attacker who knows the deployed MLC model and attempts to mimic benign telemetry via duty-cycle modulation. Quantify the AUC degradation as a function of attacker effort. This is the most important and least addressed open problem in the entire telemetry-defense literature.
8. **Drift and ageing.** Run a deployed device for 90+ days under varying ambient temperature; quantify baseline drift and develop online recalibration policies.

We estimate the first four items at 3–4 months of engineering for a single researcher with hardware access, items 5–6 at an additional 2–3 months, and items 7–8 at 6+ months. Items 1–4 alone are sufficient for a workshop-grade publication; items 1–7 are required for a top-tier conference submission.

8 Limitations

Simulation gap. The most important limitation is the obvious one: our channel-injection rules are parametric. Real attacks have temporal correlations, intermittent-execution patterns (cryptominers throttle to evade thermal envelopes), and cross-channel couplings that no parametric rule fully captures. Our results should be read as *upper bounds on the difficulty of detection*: an attack we can detect in simulation we should also detect in measurement; an attack we miss in simulation might be missed in measurement for entirely different reasons.

Symmetric channel weighting. MLC’s first term weights all channels equally after standardization. The hidden-inference result suggests learned per-channel weights, possibly from a small labelled dataset of past incidents, could close the gap with IF-LITE on focused attacks while preserving the cross-channel advantage on diffuse ones.

Linear subspace. The codeviation term uses a linear PCA basis. Non-linear coupling structures (e.g., the saturation behaviour of context-switch rate at peak load) would benefit from a non-linear autoencoder bottleneck, at the cost of more complex calibration.

Periodic and single-channel attacks. As demonstrated by the beacon result, MLC is fundamentally non-temporal: each sample is scored independently. A temporal extension—a sliding-window periodogram or LSTM scorer on residuals—is the most obvious follow-up and is also the path most likely to break through the $AUC \approx 0.58$ ceiling we observed.

Adversarial telemetry shaping. An attacker who knows the deployed MLC and is willing to slow down their attack can in principle stay within the benign envelope of every individual channel. The cross-channel correlation term raises the bar but does not eliminate the attack: a sufficiently disciplined attacker can match both the marginals and the dominant correlation subspace. The strongest defense is to randomize the channel subset MLC inspects (taking the spirit of Khasawneh *et al.* [19]) and to keep the precise channel weighting confidential.

Privacy. High-resolution telemetry collected by a fleet of edge devices can itself become a privacy concern (e.g., inference-load patterns may leak environmental information about a surveillance scene). Any production MLC deployment should aggregate features locally and ship only the scalar trust score, not the raw telemetry.

9 Conclusion

We have presented Multi-Layer Telemetry Correlation, a profile-aware passive integrity-verification framework for Edge AI systems, together with a formal threat model, a closed-form scoring rule, and a controlled simulation study that quantifies its detection power against four attack classes relative to standard unsupervised baselines. The empirical result is nuanced rather than triumphant: MLC matches or exceeds an Isolation-Forest baseline on diffuse multi-channel attacks (cryptomining, firmware-boot, beaconing) and loses slightly on focused attacks (hidden inference) where tree-based partitioning has the advantage; all telemetry-based detectors are essentially blind to single-channel periodic beacons hiding inside the variance of a high-dynamic-range channel.

We frame the work explicitly as a feasibility study. The contribution is the architecture, the formal threat model, and the honest scoping of what telemetry-only detection can and cannot do—together with a concrete experimental roadmap to a hardware-validated evaluation. The broader claim we believe the data supports is the one we opened with: as autonomous edge infrastructure proliferates, runtime integrity will require observability of *how machines physically behave while executing* alongside the existing layers of static, control-flow, and software-level attestation. MLC is one constructive step toward that broader observability.

Reproducibility. The simulation, detector implementations, and figure-generation code are released as a single ~ 500 -line numpy-only Python file (`simulate.py`) with fixed random seed and no external ML dependencies. The full experiment runs end-to-end in under 30 s on a single CPU.

Acknowledgments. The author thanks the open security and embedded-systems communities whose datasets and prior work made this study possible.

References

- [1] Tigist Abera, N. Asokan, Lucas Davi, Jan-Erik Ekberg, Thomas Nyman, Andrew Paverd, Ahmad-Reza Sadeghi, and Gene Tsudik. C-FLAT: Control-flow attestation for embedded systems software. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 743–754, 2016.
- [2] Tigist Abera, Raad Bahmani, Ferdinand Brasser, Ahmad Ibrahim, Ahmad-Reza Sadeghi, and Matthias Schunter. DIAT: Data integrity attestation for resilient collaboration of autonomous systems. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2019.
- [3] Kanad Basu, Prashanth Krishnamurthy, Farshad Khorrami, and Ramesh Karri. A survey of cyber security countermeasures using hardware performance counters. *arXiv preprint arXiv:1807.10868*, 2018.
- [4] Lejla Batina, Shivam Bhasin, Dirmanto Jap, and Stjepan Picek. CSI NN: Reverse engineering of neural network architectures through electromagnetic side channel. In *28th USENIX Security Symposium*, pages 515–532, 2019.
- [5] Alejandro Cabrera Aldaya and Billy Bob Brumley. MAD-EN: Microarchitectural attack detection through system-wide energy consumption. *arXiv preprint arXiv:2206.00101*, 2022.
- [6] Heng Cao, Yu Wang, Yangdong Liu, and Hai Wang. Detection of and countermeasure against thermal covert channel in many-core systems. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2021.
- [7] Maurantonio Caprolu, Simone Raponi, Gabriele Oligeri, and Roberto Di Pietro. Cryptomining makes noise: Detecting cryptojacking via machine learning. *Computer Communications*, 171:126–139, 2021.
- [8] Erh-Chung Chen, Pin-Yu Lee, Chun-Yu Chen, and Yi-Lun Tsai. Overload: Latency attacks on object detection for edge devices. *arXiv preprint arXiv:2304.05370*, 2023.
- [9] Shane S. Clark, Hossen Mustafa, Benjamin Ransford, Jacob Sorber, Kevin Fu, and Wenyuan Xu. WattsUpDoc: Power side channels to nonintrusively discover untargeted malware on embedded medical devices. In *USENIX Workshop on Health Information Technologies*, 2013.
- [10] John Demme, Matthew Maycock, Jared Schmitz, Adrian Tang, Adam Waksman, Simha Sethumadhavan, and Salvatore Stolfo. On the feasibility of online malware detection with performance counters. In *Proceedings of the 40th Annual International Symposium on Computer Architecture (ISCA)*, pages 559–570, 2013.
- [11] Fei Ding, Hongda Li, Feng Luo, Hongxin Hu, Long Cheng, Hai Xiao, and Rong Ge. Deep-Power: Non-intrusive and deep learning-based detection of IoT malware using power side channels. In *Proceedings of the 15th ACM Asia Conference on Computer and Communications Security (AsiaCCS)*, pages 33–46, 2020.
- [12] Vasisht Duddu, Debasis Samanta, D. Vijay Rao, and Valentina E. Balas. Stealing neural networks via timing side channels. *arXiv preprint arXiv:1812.11720*, 2018.

- [13] Sankha Baran Dutta, Hoda Naghibijouybari, Arjun Gupta, Nael Abu-Ghazaleh, Andres Marquez, and Kevin Barker. Spy in the gpu-box: Covert and side channel attacks on multi-GPU systems. *arXiv preprint arXiv:2203.15981*, 2022.
- [14] Karim Eldefrawy, Gene Tsudik, Aurélien Francillon, and Daniele Perito. SMART: Secure and minimal architecture for (establishing dynamic) root of trust. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2012.
- [15] Sebastian Garcia, Agustin Parmisano, and Maria Jose Erquiaga. IoT-23: A labeled dataset with malicious and benign IoT network traffic. Stratosphere Laboratory, Czech Technical University, 2020.
- [16] Habiba Guan et al. Runtime detection of adversarial attacks in AI accelerators using performance counters. *arXiv preprint arXiv:2503.07568*, 2025.
- [17] Kaspersky GReAT. CosmicStrand: the discovery of a sophisticated UEFI firmware rootkit. <https://securelist.com/cosmicstrand-uefi-firmware-rootkit/106973/>, 2022.
- [18] Kaspersky GReAT. MoonBounce: the dark side of UEFI firmware. <https://securelist.com/moonbounce-the-dark-side-of-uefi-firmware/105468/>, 2022.
- [19] Khaled N. Khasawneh, Nael Abu-Ghazaleh, Dmitry Ponomarev, and Lei Yu. Efficient hardware malware detectors that are resilient to adversarial evasion. In *Proceedings of the 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2018.
- [20] Khaled N. Khasawneh et al. A survey on hardware-based malware detection approaches. *arXiv preprint arXiv:2303.12525*, 2023.
- [21] Khaled N. Khasawneh, Meltem Ozsoy, Caleb Donovan, Nael Abu-Ghazaleh, and Dmitry Ponomarev. Ensemble learning for low-level hardware-supported malware detection. In *Research in Attacks, Intrusions, and Defenses (RAID)*, 2015.
- [22] Tim Koch, Daniel Tetzlaff, and Angela Bonifati. Accurate calibration of power measurements from internal power sensors on NVIDIA Jetson devices. *arXiv preprint arXiv:2306.13107*, 2023.
- [23] William Kosasih, Yusi Feng, Chitchanok Chuengsatiansup, Yuval Yarom, and Ziyuan Zhu. SoK: Can we really detect cache side-channel attacks by monitoring performance counters? In *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security (AsiaCCS)*, 2024.
- [24] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation forest. In *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM)*, pages 413–422, 2008.
- [25] X. Long et al. Lightweight autoencoder–isolation forest anomaly detection for green IoT edge gateways. *arXiv preprint arXiv:2511.18235*, 2025.
- [26] Ramya Jayaram Masti, Devendra Rai, Aanjan Ranganathan, Christoph Müller, Lothar Thiele, and Srdjan Capkun. Thermal covert channels on multi-core platforms. In *24th USENIX Security Symposium*, pages 865–880, 2015.
- [27] Yair Meidan, Michael Bohadana, Yael Mathov, Yisroel Mirsky, Asaf Shabtai, Dominik Breitenbacher, and Yuval Elovici. N-BaIoT—network-based detection of IoT botnet attacks using deep autoencoders. *IEEE Pervasive Computing*, 17(3):12–22, 2018.
- [28] Yair Meidan, Michael Bohadana, Yael Mathov, Yisroel Mirsky, Asaf Shabtai, Dominik Breitenbacher, and Yuval Elovici. N-BaIoT: Network-based detection of IoT botnet attacks using deep autoencoders. *arXiv preprint arXiv:1805.03409*, 2018.

- [29] Yan Michalevsky, Aaron Schulman, Gunaa Arumugam Veerapandian, Dan Boneh, and Gabi Nakibly. PowerSpy: Location tracking using mobile device power analysis. In *24th USENIX Security Symposium*, pages 785–800, 2015.
- [30] Yisroel Mirsky, Tomer Doitshman, Yuval Elovici, and Asaf Shabtai. Kitsune: An ensemble of autoencoders for online network intrusion detection. In *Network and Distributed System Security Symposium (NDSS)*, 2018.
- [31] Hoda Naghibijouybari, Ajaya Neupane, Zhiyun Qian, and Nael Abu-Ghazaleh. Rendered insecure: GPU side channel attacks are practical. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 2139–2153, 2018.
- [32] Alireza Nazari, Nader Sehatbakhsh, Monjur Alam, Alenka Zajic, and Milos Prvulovic. EDDIE: EM-based detection of deviations in program execution. In *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*, 2017.
- [33] NVIDIA Corporation. tegrastats utility. <https://docs.nvidia.com/jetson/archives/r35.4.1/DeveloperGuide/text/AT/TegrastatsTool.html>, 2024.
- [34] Nader Sehatbakhsh, Alireza Nazari, Haider Khan, Alenka Zajic, and Milos Prvulovic. EMMA: Hardware/software attestation framework for IoT using electromagnetic signals. In *Proceedings of the 33rd Annual Computer Security Applications Conference (ACSAC)*, 2017.
- [35] Arvind Seshadri, Mark Luk, Elaine Shi, Adrian Perrig, Leendert van Doorn, and Pradeep Khosla. Pioneer: Verifying code integrity and enforcing untampered code execution on legacy systems. In *Proceedings of the 20th ACM Symposium on Operating Systems Principles (SOSP)*, pages 1–16, 2005.
- [36] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. Toward generating a new intrusion detection dataset and intrusion traffic characterization. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, 2018.
- [37] Xueyang Wang, Charalambos Konstantinou, Michail Maniatakis, and Ramesh Karri. Con-Firm: Detecting firmware modifications in embedded systems using Hardware performance counters. In *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2015.
- [38] Boyou Zhou, Anmol Gupta, Rasoul Jahanshahi, Manuel Egele, and Ajay Joshi. Hardware performance counters can detect malware: Myth or fact? In *Proceedings of the 2018 on Asia Conference on Computer and Communications Security (AsiaCCS)*, pages 457–468, 2018.

A Pipeline diagram

The end-to-end MLC pipeline is a six-stage modular flow. Each stage hands a fixed interface to the next, so the lower analytical stages are hardware-agnostic and depend only on the channel-vector interface:

1. **Telemetry sensors.** CPU, GPU/NPU, thermal, memory, scheduler, power, latency, network channels sampled by hardware-specific drivers.
2. **Lightweight collection agent.** Built atop standard interfaces: Linux `perf`, `tegrastats`, Intel RAPL, `sysfs`.
3. **Feature extraction.** Per-window aggregation, missing-value imputation, and per-channel standardization.

4. **Mode classifier.** Lightweight classifier selects the active operating profile $m^* \in \mathcal{M}$ for the current window.
5. **MLC scoring.** Computes $\Phi(\mathbf{x}_t) = \min_m [\phi_m^{(z)} + \lambda \phi_m^{(\text{codev})}]$ as defined in Equations (1)–(2).
6. **Threshold and response.** Compares Φ to a site-specific threshold; emits an alert or a downgraded-trust handoff to the controller.